

MÓDULO 2
**Arquitecturas de seguridad
y técnicas**
Ciberseguridad

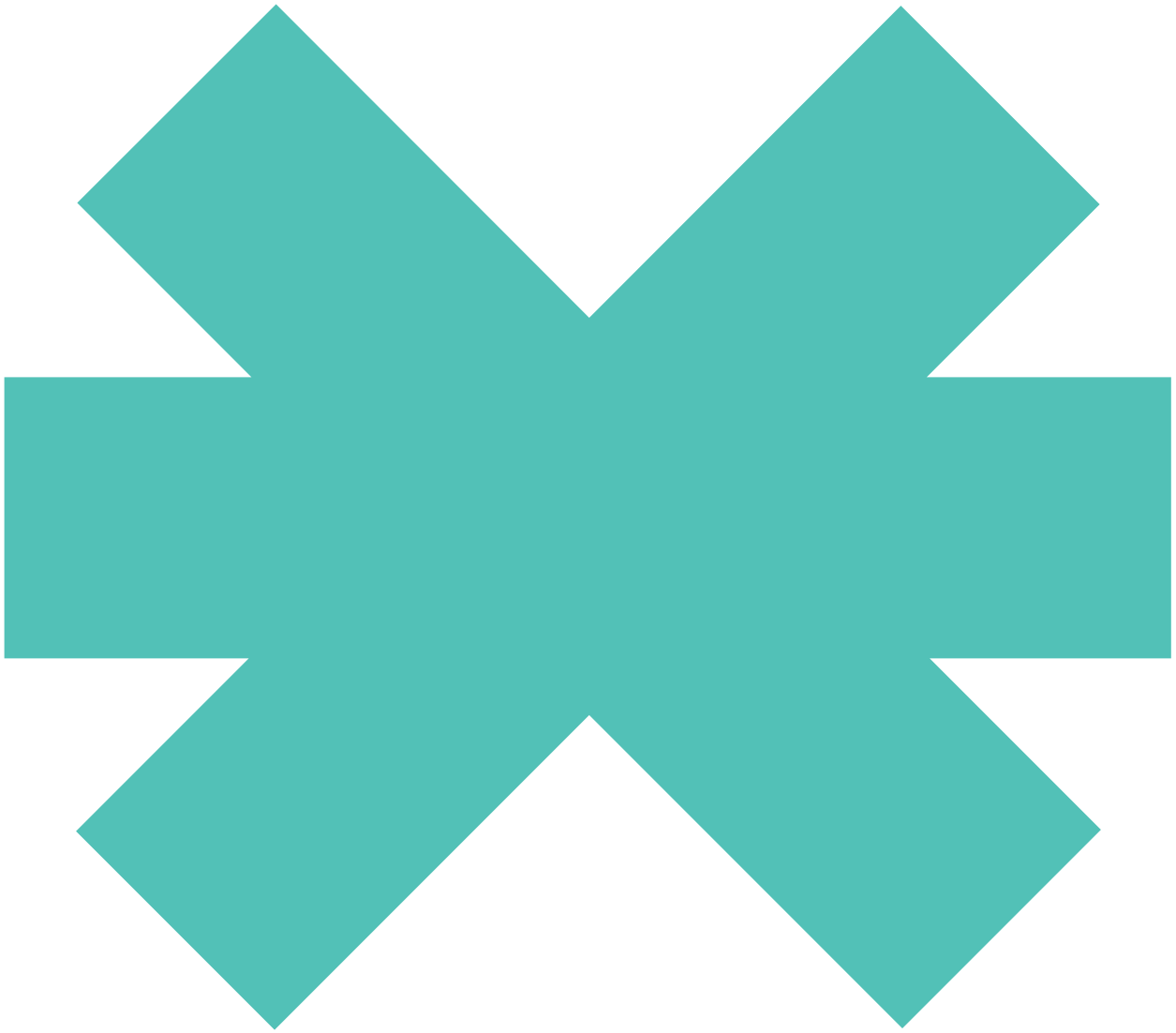
4

Despliegue de servidores proxy



New
Technology
School

Tokio.



4 Despliegue de servidores proxy

Sumario

4.1	Servidores <i>proxy</i>	146
4.2	Tipos de <i>proxies</i>	148
4.3	Arquitectura y componentes	149
4.4	Configuración	150

4.1 Servidores *proxy*

Denominamos *proxy* a un **intermediario** que se instala entre el emisor y el receptor para poder recibir las peticiones. En el ámbito informático, tan solo se trata de un punto situado entre un usuario e internet que permite dotar de **anonimato** a las conexiones, tanto las que se producen desde dentro hacia fuera como las que se generan desde fuera hacia dentro.

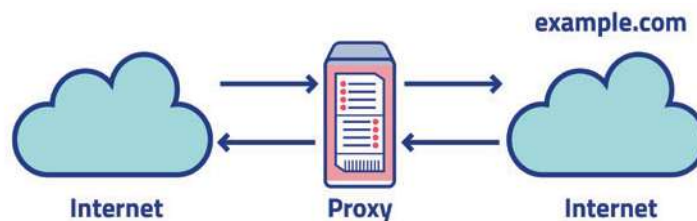


Figura 8

Tal y como se muestra en la imagen anterior, si un usuario se conecta directamente desde un dispositivo con conexión a internet, la máquina desde la que realice la petición identificará la IP de la que procede; en cambio, si dicha petición se efectúa a través del *proxy*, la IP será la de este, de manera que, en cierta forma, se mantiene el anonimato del emisor, es decir, de quien realmente está realizando la petición.

Normalmente, los *proxies* se utilizan para acceder a páginas desde una geolocalización diferente a la real por si estas presentan restricciones o rechazan conexiones desde nuestro país de origen.

No debemos confundir un *proxy*, que dota de anonimato a nuestra IP, con una VPN (*Virtual Private Network*), teóricamente más segura.

Existen *proxies* gratuitos que podemos configurar para evitar que nuestra IP resulte visible, por ejemplo:

- **ProxySite:** <https://www.proxysite.com/es/>
- **CroxyProxy:** https://www.croxyproxy.com/_es/
- **Proxyium:** <https://proxyium.com/>
- **Hide.me:** <https://hide.me/es/>

Adicionalmente, se puede realizar una configuración directa desde el sistema operativo que también podemos llevar a cabo en dispositivos móviles, consiguiendo que desempeñe la misma función que si nos conectásemos desde un ordenador.

Usar un *proxy* en una red para establecer conexiones entre el exterior (internet) y el interior (cada dispositivo de esa red) posee múltiples ventajas:

- **Mejora la privacidad.** El *proxy* actúa como intermediario entre el cliente y el servidor, ocultando la dirección IP del cliente real y proporcionando anonimato.

- **Filtrado más eficiente.** Los *proxies* pueden utilizarse para implementar filtros de contenido en la red, bloqueando el acceso a aquel no deseado o malicioso, así como a determinados sitios web.
- **Mejor rendimiento y caché.** Los recursos solicitados con mayor frecuencia, como páginas web o archivos multimedia, se pueden almacenar en caché, lo que permite acceder más rápidamente a los recursos, ya que se solicitan directamente al *proxy* y no al servidor original.
- **Acceso a contenido georrestringido.** Al enmascarar la ubicación del usuario, el *proxy* puede proporcionar acceso a contenido restringido geográficamente.
- **Mayor seguridad y control de tráfico.** Al implementar el *proxy*, pueden establecerse políticas y reglas para controlar todo el tráfico de la red, incluyendo el bloqueo de ciertos tipos no autorizados.
- **Monitoreo.** Asimismo, el *proxy* puede registrar y supervisar el tráfico que pasa por él, lo que permite analizarlo e identificar posibles amenazas.

Junto con las ventajas mencionadas, los *proxies*, también pueden presentar algunas desventajas:

- **Sobrecarga de rendimiento.** El uso del *proxy* puede suponer una sobrecarga adicional en la red, ya que todos los datos han de pasar por este antes de llegar al destino.
- **Punto único de fallo.** Si el *proxy* falla o no permanece disponible, puede interrumpirse el acceso a internet o a los servicios que dependen de él.
- **Posible vulnerabilidad.** Del mismo modo, si el *proxy* no se encuentra correctamente configurado, puede convertirse en un punto de entrada para ataques cibernéticos.
- **Limitaciones de privacidad.** Si bien los *proxies* proporcionan anonimato, este no es completo, ya que pueden almacenar registros de actividad.
- **Problemas de compatibilidad.** Algunos servicios y aplicaciones pueden ser incompatibles con el uso de un *proxy*.
- **Desactualización.** En algunos *proxies*, la información más actual puede verse afectada.

4.2 Tipos de *proxies*

Como hemos visto, los *proxies* pueden representar componentes clave en el ámbito de las redes y de la seguridad al actuar como intermediarios entre cliente y servidor, permitiendo que los usuarios y las usuarias accedan a recursos en línea. Podemos referirnos a varios tipos de *proxies* con características y funcionalidades específicas:

- **Proxy web.** Los usuarios y las usuarias pueden acceder de forma manual desde el navegador y a través de una aplicación web a un servidor *proxy* HTTP que funciona como intermediario y que recibe la petición mediante una URL, accediendo al servidor de la web solicitada y devolviendo el contenido a una página propia. Además, ofrece funciones como caché, filtrado de contenido y control de acceso.
- **Proxy inverso.** Se trata de un servidor que se sitúa delante de los servidores web, a los que reenvía las solicitudes del cliente (por ejemplo, del navegador web). De este modo, mejora el rendimiento, la escalabilidad y la seguridad de los servicios web.

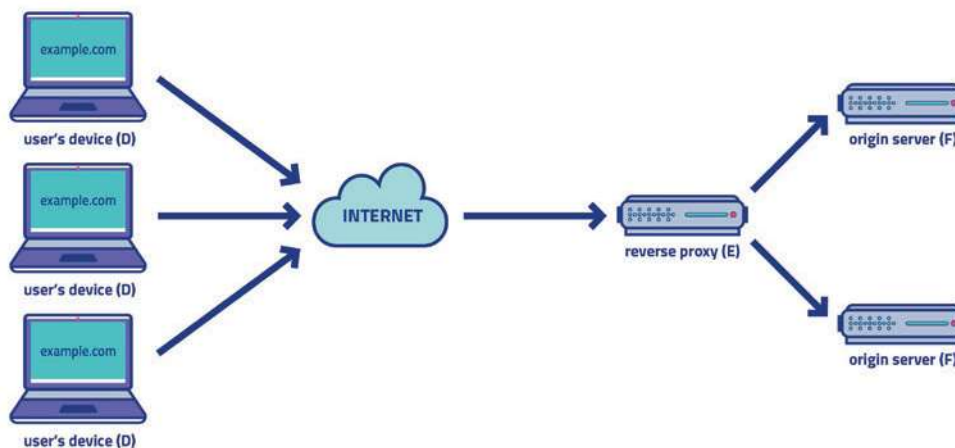


Figura 9. Representación de un *proxy* inverso

- **Proxy NAT (*Network Address Translation*).** Permite que múltiples dispositivos compartan una única dirección IP. En este sentido, traduce a una dirección IP pública las ubicaciones privadas de los dispositivos cuando se comunican con recursos en internet, protegiendo la red interna detrás del *proxy*.
- **Proxy interdominio.** También conocido como *proxy* de dominio cruzado, lo utilizan las tecnologías web asíncronas (Flash, AJAX, Comet, etc.), que presentan restricciones para establecer una comunicación entre elementos localizados en distintos dominios. Por tanto, facilita la comunicación entre diferentes dominios o redes separadas.
- **Proxy abierto.** Se trata de un *proxy* público al que puede acceder cualquier persona. A diferencia de los *proxies* privados, que se configuran y se usan dentro de una organización o red específica, los *proxies* abiertos permanecen disponibles para su uso público y pueden proporcionar anonimato y acceso a contenido georrestringido.

4.3 Arquitectura y componentes

La arquitectura requerida de cara al despliegue del servidor *proxy* implican el trabajo conjunto de varios componentes para facilitar el enrutamiento y la comunicación entre los clientes y los servidores de destino. A continuación, se detallan los principales:

- **Cliente.** Es el dispositivo o la aplicación desde donde se origina la solicitud al servidor *proxy*. Puede tratarse de una computadora, de un dispositivo móvil o de cualquier otro que necesite acceder a recursos a través del servidor *proxy*.
- **Servidor *proxy*.** Actúa como intermediario entre el cliente y el servidor de destino. Cuando un cliente envía una solicitud, el servidor *proxy* la intercepta y la reenvía al servidor de destino en nombre del cliente.
- **Servidor de destino.** Servidor final al que se dirigen las solicitudes del cliente. Puede tratarse de un servidor web, de un servidor de aplicaciones o cualquier otro tipo que proporcione los recursos o servicios solicitados por el cliente.
- **Configuración de red y enrutamiento.** El *proxy* debe situarse estratégicamente en la red para dirigir el tráfico de los clientes hacia los servidores de destino. Una configuración de red y un enrutamiento adecuados resultan fundamentales para garantizar que las solicitudes del cliente sean correctamente redirigidas al *proxy* y, posteriormente, al servidor de destino.
- **Protocolos utilizados.** Pueden trabajar con diversos protocolos de red, como HTTP, HTTPS, SOCKS, FTP, etc.; dependerá del tipo de *proxy* y de las necesidades específicas de la aplicación o del entorno en el que se despliegue.
- **Consideraciones de seguridad.** A la hora de desplegar un servidor *proxy*, debemos tener en cuenta ciertos aspectos de seguridad para proteger la red y los datos sensibles, entre ellos, incluir autenticación y autorización para controlar el acceso al servidor, cifrado de comunicaciones, políticas de filtrado y control de acceso, etc.

Debemos tener en cuenta que la arquitectura y los componentes pueden variar en función del tipo de servidor *proxy* utilizado y de los requisitos específicos de implementación. Algunas soluciones pueden ofrecer características más avanzadas, como la inspección de paquetes, la manipulación de contenido y la optimización de la red, mientras que otras se orientan a funciones más básicas.

4.4 Configuración

Una de las fases más importantes a la hora de implementar un servidor *proxy* es su correcta configuración, de manera que se pueda garantizar un funcionamiento adecuado y seguro.

En este sentido, se encuentran disponibles varias opciones de *software* y de herramientas, por ejemplo, Squid, Nginx, Apache o HAProxy; la elección dependerá de los requisitos específicos del entorno, del nivel de funcionalidad deseado y de la compatibilidad con los protocolos requeridos.

Una vez seleccionado el *software* del servidor *proxy*, tendremos que configurar los parámetros necesarios, como la especificación del puerto en el que recibirá las solicitudes, la caché, la configuración de reglas de filtrado y control de acceso o la configuración de la autenticación y autorización. También habrá que implementar mecanismos de monitoreo y registro para supervisar la actividad del servidor *proxy*, lo que nos permitirá analizar su rendimiento e identificar posibles intentos de acceso no autorizados o actividades sospechosas.

Finalmente, antes de implementar del todo el servidor, será necesario realizar pruebas para verificar que funciona correctamente y para asegurarnos de que reúne los requisitos establecidos, de manera que podamos ajustar y optimizar la configuración para mejorar su rendimiento.

Consideraciones de seguridad y mitigación de riesgos

Como ya hemos señalado, si bien los servidores *proxy* garantizan el anonimato, en ocasiones pueden suponer una brecha de seguridad, pues al almacenar registros de actividad, la privacidad no es completa. Por consiguiente, resulta necesario considerar ciertos aspectos para mejorar la seguridad:

- **Autenticación y autorización.** La implementación de estas medidas permitirá verificar la identidad de aquellos usuarios/as o dispositivos que traten de acceder al servidor *proxy*, así como los recursos y las acciones permitidas para cada uno de ellos.
- **Mitigación de riesgos.** Debemos mantener el *software* actualizado con los últimos parches de seguridad, deshabilitar los servicios innecesarios, configurar adecuadamente las políticas de seguridad y restringir el acceso no necesario al servidor. Igualmente, se pueden implementar medidas adicionales frente a ataques comunes recurriendo a cortafuegos y a sistemas de prevención de intrusiones.
- **Filtrado y control de acceso.** Definiremos reglas que permitan o denieguen el acceso a recursos específicos o a sitios web en función de los criterios que establezcamos, como direcciones IP, URL, palabras clave, tipo de contenido, etc.
- **Supervisión.** Además, implementaremos sistemas de monitoreo dirigidos a la recolección y análisis de un registro de eventos, de manera que podamos detectar cualquier actividad sospechosa.
- **Privacidad y cifrado.** Finalmente, resulta fundamental garantizar la privacidad durante la comunicación asegurándonos de que se utilicen certificados SSL/TLS válidos y configurando adecuadamente las políticas de cifrado.

Si implementamos estas medidas de seguridad, podremos garantizar un despliegue más seguro del servidor *proxy*, protegiendo la red, los recursos y los usuarios frente a amenazas potenciales.

Nginx, el *proxy* más conocido

Existen múltiples productos comerciales conocidos que desempeñan las mismas funciones que cualquier tipo de *proxy*; no obstante, nos centraremos en el más popular de todos hasta el momento, **Nginx**.

Nginx es un servidor web de código abierto, es decir, de carácter gratuito, que ha evolucionado tras su éxito inicial, incorporando nuevas funcionalidades como *proxy* inverso, balanceador de carga o caché HTTP.

De hecho, utilizan Nginx grandes compañías como Google, GitLab, VMWare, LinkedIn, Facebook o Twitter, lo que demuestra el potencial y el buen funcionamiento de esta tecnología tan extendida mundialmente.

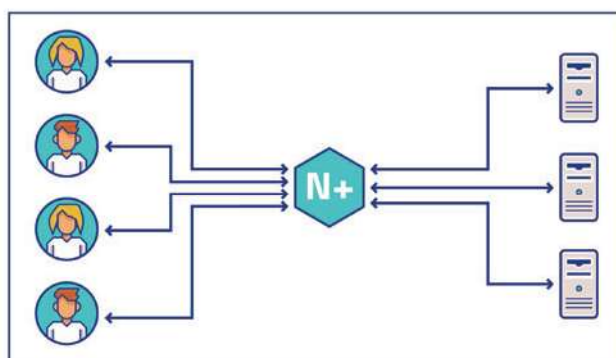


Figura 10

Nginx fue creado como un **software libre** por Igor Sysoev en octubre de 2004 con el propósito de resolver el **problema de C10K**, esto es, cómo optimizar las conexiones de red para gestionar un gran número de clientes simultáneos, en concreto, 10.000 peticiones concurrentes.

Se trata de un servidor que presenta un reducido gasto de memoria y una elevada concurrencia y cuyo objetivo consiste en **crear varias peticiones en un solo hilo**, de manera que no realiza solicitudes diferentes al servidor web, provocando que se descargue de manera notable.

Veamos las principales características de Nginx:

- *Proxy* inverso con caché.
- IPv6.
- Balanceo de carga.
- Soporte FastCGI con almacenamiento en caché.
- *Websockets*.
- Manejo de archivos estáticos, archivos de índice y autoindexación.
- TLS / SSL con SNI.

Existe una considerable competencia entre los distintos servidores a la hora de ofrecer mejores características para las aplicaciones que se van a desplegar. En este sentido, realizaremos una comparativa con Apache que, sin duda, es uno de los servidores webs más extendidos:

- **Compatibilidad del sistema operativo.** En este aspecto, señalaríamos un empate, ya que ambos son compatibles con Linux. Con todo, si nos centramos en Windows, Nginx no funcionaría tan bien como Apache.
- **Soporte al usuario.** En internet, se encuentra disponible documentación relativa a ambos servidores, incluso una comunidad específica en Stack Overflow. Sin embargo, Apache no ofrece soporte por parte de la compañía, lo que proporciona ventaja a Nginx en este aspecto.
- **Rendimiento.** Nginx puede ejecutar simultáneamente miles de conexiones y es el doble de rápido que Apache, además de utilizar menos memoria. No obstante, si se trata de ejecución dinámica, ambos servidores cuentan con la misma capacidad, si bien Nginx funciona mejor para páginas más estáticas.